

migrator.jar y estructura del repositorio

Guía técnica v2 con un ejemplo adicional para cada carpeta, módulo y paquete

Propósito de esta revisión

Mantener la explicación técnica anterior y añadir ejemplos concretos que muestren cómo se usaría cada parte del repositorio. Los ejemplos planificados están marcados para no confundir diseño futuro con código ya implementado.

Proyecto: C:\cosme\LogixoneJakarta11

Baseline: Java 21, Jakarta EE 11, WildFly 41, PostgreSQL 18.4 y Docker

Versión del documento: 2 - ejemplos adicionales

Estado: Sprint 1 - infraestructura y migrador one-shot validados

Documento de arquitectura, operación y aprendizaje

Contenido

1. Qué es migrator.jar
2. Cómo participa en el arranque
3. Idempotencia, checksums, configuración y seguridad
4. migrator.jar frente a logixone.war
5. Mapa general y convención de ejemplos
6. Herramientas, plataforma y capas del kernel
7. Paquetes internos del migrador
8. Plugins, distribución y pruebas
9. Infraestructura, documentación y archivos generados
10. Estado verificado y siguiente paso

Cómo leer los ejemplos

Cada tarjeta presenta un ejemplo base y un Ejemplo adicional (nuevo). Estado Actual significa que hay archivos o comportamiento verificable; Esqueleto o planificado describe cómo se usará el módulo en historias posteriores.

Convenciones

- Las rutas son relativas a la raíz del repositorio salvo que se indique una ruta dentro del contenedor.
- Los ejemplos futuros son diseño orientativo y todavía requieren código y pruebas en su historia correspondiente.
- Los secretos y cachés locales pertenecen a .tools y nunca deben entrar en Git, imágenes o artefactos.

1. Qué es migrator.jar

migrator.jar es un programa Java ejecutable e independiente que prepara y valida PostgreSQL antes de que el ERP pueda arrancar. No es un servidor: inicia, trabaja y termina.

Ubicaciones del artefacto

```
Build local:
migrator/target/migrator-0.1.0-SNAPSHOT-executable.jar
Dentro de Docker:
/opt/logixone/migrator.jar
```

Qué contiene

- Siete clases de producción del paquete `py.com.logixone.migrator`.
- Flyway Core 12.8.1 y el módulo PostgreSQL de Flyway.
- Driver pgJDBC 42.7.10, migraciones SQL y descriptores ServiceLoader.
- Avisos LICENSE y NOTICE agregados por Maven Shade.

Qué no contiene

- No incluye `logixone.war` ni ejecuta WildFly.
- No atiende HTTP y no permanece activo después de migrar.
- No incorpora contraseñas, `.tools`, `workspace` del builder ni JUnit.

Ejemplo práctico

Si PostgreSQL ya tiene V1 y la imagen nueva contiene V2, el migrador valida el checksum de V1, aplica solamente V2, valida el resultado y termina con código 0.

2. Cómo participa en el arranque

Compose expresa el orden como una dependencia verificable. La aplicación no puede iniciar solo porque el contenedor del migrador exista; debe esperar que termine correctamente.

Paso	Componente	Condición	Resultado
1	postgres	Inicia con volumen y secreto	Conserva datos entre recreaciones
2	health check	pg_isready responde	PostgreSQL queda saludable
3	migrator	Flyway valida y migra	Código 0, 1 o 2
4	Compose	service_completed_successfully	Código 0 habilita app
5	app	WildFly inicia	ERP atiende solo tras migración válida

Secuencia detallada

- postgres usa una imagen PostgreSQL 18.4 fijada por digest y monta postgres-data.
- migrator recibe URL, usuario y ruta del secreto /run/secrets/postgres_password.
- Flyway crea o valida su historial, compara checksums y aplica migraciones pendientes.
- Código 0 habilita app; código 1 o 2 mantiene la aplicación bloqueada.

Ejemplo adicional

Si V1 fue editada después de aplicarse, Flyway detecta el checksum distinto. migrator termina con código 1 y Compose no arranca app, evitando ejecutar el ERP sobre un esquema dudoso.

3. Idempotencia, checksums, configuración y seguridad

Idempotencia e inmutabilidad

El proceso puede repetirse. Flyway consulta el historial y no vuelve a ejecutar una migración aplicada. Una corrección se agrega como V2 o posterior; una migración ya aplicada no se edita.

Primera ejecución: migrations_executed=1 schema_version=1

Segunda ejecución: migrations_executed=0 schema_version=1

Contrato de configuración

Variable	Uso	Regla
LOGIXONE_DB_URL	URL JDBC	Debe comenzar con jdbc:postgresql://
LOGIXONE_DB_USER	Usuario	Obligatorio y no vacío
LOGIXONE_DB_PASSWORD_FILE	Ruta del secreto	Se lee desde archivo externo

El secreto se rechaza si falta, está vacío, usa varias líneas, supera 4096 bytes o no es UTF-8. Su contenido no se incorpora a logs, excepciones documentadas ni toString.

Códigos de salida

Código	Significado	Consecuencia
0	Migración y validación correctas	Compose puede iniciar app
1	Fallo Flyway, conexión, SQL o checksum	Compose bloquea app
2	Configuración ausente o inválida	Corregir variables o secreto

4. migrator.jar frente a logixone.war

Aspecto	migrator.jar	logixone.war
Responsabilidad	Preparar y validar la base	Ejecutar el ERP y atender solicitudes
Proceso	Java independiente one-shot	Aplicación Jakarta EE en WildFly
Duración	Termina después de migrar	Permanece ejecutándose
Entrada	Variables externas y secreto	Configuración de aplicación y servidor
Salida	Código 0, 1 o 2	HTTP, UI, APIs y procesamiento
Mutación de esquema	Sí, mediante migraciones	No; JPA valida el esquema

Por qué están separados

Un proceso separado define con claridad el orden, el fallo y la recuperación. Varias réplicas web no intentan migrar al mismo tiempo y el servidor nunca arranca sobre una base no validada.

Artefactos verificados

- JAR ejecutable SHA-256: 56343D2FD82E42FD53F341BC32155A7122D20C30AC342FC7C9653A778A846091.
- WAR SHA-256: 23C935CEB30AB75CCDD72A9CF96658E6A0B63B50B4A552AE908D7B7F1488BD39.

5. Mapa general y convención de ejemplos

```
LogixoneJakarta11/  
  .mvn/ .tools/  
  platform-bom/ plugin-api/  
  kernel-api/ kernel-domain/  
  kernel-application/ kernel-infrastructure-jakarta/  
  web-shell/ migrator/  
  plugins/ distribution/  
  tests/ infra/  
  docs/ target/ (dentro de cada módulo)
```

El POM padre coordina 13 módulos hijos; con el padre forman un reactor de 14 proyectos. La arquitectura es un monolito modular: un único WAR con límites explícitos entre contratos, dominio, casos de uso, infraestructura y plugins.

Qué cambió en esta versión del PDF

- Cada tarjeta conserva un ejemplo base para explicar el propósito.
- Cada tarjeta añade un Ejemplo adicional (nuevo), resaltado en verde.
- Se indica si el ejemplo corresponde al estado actual o a una historia futura.

Regla de interpretación

Un ejemplo planificado no afirma que la clase ya exista. Sirve para explicar dónde debe ubicarse esa responsabilidad cuando se implemente, manteniendo los límites de arquitectura.

6. Herramientas, plataforma y capas del kernel

.mvn/ Actual	
Responsabilidad	Configura Maven Wrapper y permite construir con una versión controlada de Maven.
Ejemplo base	mvnw.cmd -v descarga o reutiliza Maven 3.9.16 sin depender de una instalación global.
Ejemplo adicional (nuevo)	En un agente CI limpio, .mvn/wrapper/maven-wrapper.properties hace reproducible el mismo comando verify.

.tools/ Actual	
Responsabilidad	Aloja herramientas, descargas, cachés, secretos y temporales locales que no entran en Git ni Docker.
Ejemplo base	.tools/jdk conserva el JDK 21 local y .tools/maven-repository las dependencias Maven.
Ejemplo adicional (nuevo)	.tools/secrets/postgres-password.txt entrega la contraseña a Compose sin incorporarla al YAML ni a una imagen.

platform-bom/ Esqueleto / ejemplo planificado	
Responsabilidad	Centraliza versiones compatibles para impedir que cada módulo elija dependencias diferentes.
Ejemplo base	Una versión común de Jakarta EE evita discrepancias entre kernel, shell y plugins.
Ejemplo adicional (nuevo)	El BOM podrá fijar Testcontainers y REST Assured una sola vez para todos los módulos de integración.

plugin-api/ Esqueleto / ejemplo planificado	
Responsabilidad	Define el SPI público y Java puro que describe identidad, compatibilidad, dependencias y contribuciones de plugins.
Ejemplo base	PluginDescriptor puede exponer id, nombre, versión y capacidades sin importar jakarta.*.
Ejemplo adicional (nuevo)	PluginDependency puede declarar que ventas requiere inventario >= 1.2 antes de permitir la activación.

kernel-api/ Esqueleto / ejemplo planificado	
Responsabilidad	Publica contratos transversales del núcleo que pueden consumir shell, infraestructura y plugins.
Ejemplo base	CompanyContext identifica la empresa activa sin revelar una entidad JPA.
Ejemplo adicional (nuevo)	AuthorizationPort permite preguntar si un usuario posee ventas.factura.emitir sin conocer la implementación de seguridad.

kernel-domain/		Esqueleto / ejemplo planificado
Responsabilidad	Contiene reglas de negocio puras del núcleo: empresas, activación, compatibilidad y dependencias.	
Ejemplo base	PluginActivation impide habilitar un plugin incompatible con la versión del kernel.	
Ejemplo adicional (nuevo)	DependencyGraph rechaza un ciclo A -> B -> A antes de persistir cualquier activación.	

kernel-application/		Esqueleto / ejemplo planificado
Responsabilidad	Coordina casos de uso del núcleo mediante puertos, sin depender de HTTP, JPA o JSF.	
Ejemplo base	ActivatePluginUseCase valida permisos, compatibilidad y dependencias antes de activar.	
Ejemplo adicional (nuevo)	ListAvailableCapabilitiesUseCase combina registro y contexto de empresa para devolver solo capacidades habilitadas.	

kernel-infrastructure-jakarta/		Esqueleto / ejemplo planificado
Responsabilidad	Implementa adaptadores Jakarta EE: CDI, JPA, JTA, seguridad, auditoría y descubrimiento.	
Ejemplo base	JpaPluginActivationRepository implementará el puerto de activaciones usando el esquema core.	
Ejemplo adicional (nuevo)	CdiPluginDiscovery podrá localizar implementaciones del SPI empaquetadas en WEB-INF/lib sin classloaders personalizados.	

web-shell/		Esqueleto / ejemplo planificado
Responsabilidad	Proporciona la carcasa web común: sesión, navegación, menú y composición de contribuciones.	
Ejemplo base	MenuAssembler omite entradas aportadas por plugins desactivados para la empresa actual.	
Ejemplo adicional (nuevo)	PluginPageGuard devuelve acceso denegado si una URL pertenece a una capacidad no habilitada, aunque se invoque directamente.	

7. Paquetes internos del migrador

migrator/ Actual	
Responsabilidad	Construye el proceso Java one-shot que migra y valida PostgreSQL antes de iniciar WildFly.
Ejemplo base	migrator-0.1.0-SNAPSHOT-executable.jar incluye Flyway, pgJDBC y el punto de entrada MigratorMain.
Ejemplo adicional (nuevo)	Una imagen nueva puede incorporar V2__add_plugin_registry.sql y aplicar solo esa versión sobre una base que ya posee V1.

py.com.logixone.migrator (producción) Actual	
Responsabilidad	Agrupar las siete clases que leen configuración, ejecutan Flyway, representan el resultado y fijan el código de salida.
Ejemplo base	MigratorConfiguration lee LOGIXONE_DB_URL, LOGIXONE_DB_USER y la ruta del secreto.
Ejemplo adicional (nuevo)	MigrationOutcome permite informar schema_version y migrations_executed sin registrar la contraseña ni la URL completa.

db/migration/core/ Actual + ejemplo futuro	
Responsabilidad	Contiene migraciones SQL versionadas e inmutables que son propiedad del kernel.
Ejemplo base	V1__initialize_core_schema.sql crea core.system_metadata y registra schema_owner=core.
Ejemplo adicional (nuevo)	Una futura V2__create_plugin_activation.sql añadiría una tabla nueva sin editar el checksum de V1.

py.com.logixone.migrator (pruebas) Actual	
Responsabilidad	Verifica configuración, códigos de salida y recursos SQL sin incorporar JUnit al JAR ejecutable.
Ejemplo base	MigratorConfigurationTest rechaza un secreto inexistente, vacío, multilínea o demasiado grande.
Ejemplo adicional (nuevo)	MigratorCommandTest puede simular un fallo de checksum y comprobar que el proceso devuelve código 1 y no habilita app.

migrator/target/ Actual / generado	
Responsabilidad	Reúne artefactos reconstruibles del módulo y reportes de pruebas; nunca es código fuente.
Ejemplo base	migrator-0.1.0-SNAPSHOT-executable.jar es el artefacto copiado como /opt/logixone/migrator.jar.
Ejemplo adicional (nuevo)	surefire-reports/ permite revisar qué clase de prueba falló sin abrir el JAR de producción.

8. Plugins, distribución y pruebas

plugins/reference-plugin/		Esqueleto / ejemplo planificado
Responsabilidad	Será el plugin demostrativo que prueba el contrato completo sin introducir todavía un dominio ERP real.	
Ejemplo base	ReferencePluginDescriptor declarará una capacidad reference.view y una entrada de menú.	
Ejemplo adicional (nuevo)	Una prueba adicional construirá la distribución sin reference-plugin para demostrar que el kernel no depende de su implementación.	

distribution/logixone-war/		Actual + composición futura
Responsabilidad	Ensambla físicamente kernel, infraestructura, shell y plugins seleccionados en un único WAR.	
Ejemplo base	WEB-INF/lib contiene los JAR de plugin-api, kernel y web-shell contruidos por el reactor.	
Ejemplo adicional (nuevo)	Un perfil Maven podrá incluir reference-plugin para pruebas y excluirlo para verificar que el WAR sigue compilando.	

tests/architecture-tests/		Esqueleto / ejemplo planificado
Responsabilidad	Alojará reglas ArchUnit para proteger las dependencias y responsabilidades entre capas.	
Ejemplo base	Una regla impedirá que kernel-domain importe jakarta.persistence.Entity.	
Ejemplo adicional (nuevo)	Otra regla impedirá que plugin-api dependa de WildFly, JSF, Hibernate o clases internas de un plugin.	

tests/integration-tests/		Esqueleto / ejemplo planificado
Responsabilidad	Probará PostgreSQL real, migraciones, JPA, CDI y contratos entre kernel y plugins.	
Ejemplo base	Testcontainers podrá iniciar PostgreSQL, aplicar V1 y verificar core.system_metadata.	
Ejemplo adicional (nuevo)	Una segunda prueba reiniciará el contenedor con el mismo volumen y verificará que Flyway ejecuta cero migraciones pendientes.	

tests/e2e-tests/		Esqueleto / ejemplo planificado
Responsabilidad	Probará recorridos completos desde el navegador cuando exista una interfaz navegable.	
Ejemplo base	Playwright comprobará que el menú del plugin aparece al activarlo para una empresa.	
Ejemplo adicional (nuevo)	Otro escenario abrirá directamente la URL del plugin desactivado y esperará acceso denegado, no solo un menú oculto.	

9. Infraestructura, documentación y archivos generados

infra/compose/ Actual	
Responsabilidad	Declara servicios, red interna, volumen, secreto, health checks y orden de arranque.
Ejemplo base	app espera a postgres saludable y a migrator completado con código 0.
Ejemplo adicional (nuevo)	compose.env.example permite cambiar puerto o etiqueta local sin alterar la topología común de desarrollo, pruebas y producción.

infra/docker/ Actual	
Responsabilidad	Define builds multi-stage reproducibles para la aplicación y el migrador, con bases fijadas por digest.
Ejemplo base	Dockerfile.migrator construye con Temurin JDK 21 y ejecuta con Temurin JRE 21 como usuario 10001.
Ejemplo adicional (nuevo)	Dockerfile de app copia únicamente logixone.war al runtime WildFly y deja .tools, target intermedios y secretos fuera de la imagen.

docs/adr/ Actual	
Responsabilidad	Registra decisiones arquitectónicas, alternativas, consecuencias y estado.
Ejemplo base	0004-docker-iac-promocion-digest.md fija la promoción de una misma imagen por digest.
Ejemplo adicional (nuevo)	Un ADR futuro deberá justificar cualquier decisión de carga dinámica de plugins antes de introducir OSGi o classloaders propios.

docs/architecture/ Actual	
Responsabilidad	Describe el diseño vigente, límites de módulos, flujos y estrategia de pruebas.
Ejemplo base	overview.md explica el monolito modular y la dirección permitida de dependencias.
Ejemplo adicional (nuevo)	test-strategy.md podrá añadir una matriz que relacione cada contrato de plugin con prueba unitaria, ArchUnit e integración.

docs/backlog/ Actual	
Responsabilidad	Mantiene épicas, historias, criterios de aceptación y trabajo priorizado.
Ejemplo base	Una historia puede exigir rechazar dependencias cíclicas entre plugins.
Ejemplo adicional (nuevo)	Otra historia puede definir aceptación para activar un plugin por empresa sin eliminar sus datos al desactivarlo.

docs/evidence/ Actual	
Responsabilidad	Conserva resultados reproducibles de builds, Docker, migraciones y controles documentales.
Ejemplo base	J11-S1-03-migrator-one-shot.md documenta la primera migración, reejecución y fallo de checksum.
Ejemplo adicional (nuevo)	La evidencia de este PDF registra páginas, hash, extracción de texto, renderizado e inspección visual.

docs/knowledge-base/ Actual / pendiente de poblar	
Responsabilidad	Captura reglas observadas en el legado como conocimiento, no como código copiado.
Ejemplo base	Un documento puede describir cómo se asignan permisos actuales a opciones de menú.
Ejemplo adicional (nuevo)	Otro documento puede caracterizar el cierre de una factura con entradas, salidas, excepciones y casos límite antes de diseñar el plugin.

docs/runbooks/ Actual	
Responsabilidad	Ofrece procedimientos operativos reproducibles para construir, ejecutar, diagnosticar y recuperar.
Ejemplo base	migrator.md explica variables, secreto, ejecución y códigos de salida.
Ejemplo adicional (nuevo)	compose.md puede guiar una recuperación tras código 1: revisar logs, corregir la nueva migración y relanzar sin usar Flyway repair para ocultar el fallo.

docs/sprints/ Actual	
Responsabilidad	Registra objetivos, historias, avances, resultados y cierre de cada Sprint.
Ejemplo base	sprint-01/J11-S1-03-docker-iac.md contiene criterios para Docker, Compose y migrador.
Ejemplo adicional (nuevo)	La retrospectiva del Sprint podrá comparar lo planificado con evidencias 14/14 y anotar riesgos para J11-S1-04.

docs/output/pdf/ Actual	
Responsabilidad	Contiene documentos PDF generados, verificables y destinados a consulta o distribución.
Ejemplo base	explicacion-migrator-y-estructura-repositorio.pdf reúne esta guía en un archivo estable.
Ejemplo adicional (nuevo)	Una versión futura puede regenerarse tras J11-S1-04 y conservar en evidencia su nuevo SHA-256 y fecha de corte.

target/ Actual / generado	
Responsabilidad	Contiene resultados generados por Maven y puede reconstruirse por completo.
Ejemplo base	distribution/logixone-war/target/logixone.war es el desplegable actual.
Ejemplo adicional (nuevo)	migrator/target/surefire-reports guarda resultados XML y texto útiles para diagnosticar una prueba sin versionarlos como fuente.

Archivos de gobierno en la raíz		Actual
Responsabilidad	pom.xml gobierna el reactor; AGENTS.md fija reglas; mvnw y mvnw.cmd ejecutan Maven; archivos ignore controlan contenido local.	
Ejemplo base	pom.xml lista los 13 módulos hijos y centraliza versiones de dependencias y plugins Maven.	
Ejemplo adicional (nuevo)	.dockerignore evita que .tools, .git, target y documentación entren al contexto de construcción de las imágenes.	

10. Estado verificado y siguiente paso

Estado técnico al 2026-07-23

- El migrador tiene 8 pruebas verdes y el reactor completo 14/14.
- La primera ejecución aplicó V1; las siguientes aplicaron cero cambios.
- Un checksum inválido produjo código 1 y bloqueó app.
- El volumen logixone_postgres-data se preservó después de docker compose down.
- Los módulos de kernel, shell, plugin de referencia y pruebas continúan como esqueletos intencionales.

Cobertura de ejemplos de esta revisión

Grupo	Elementos explicados	Ejemplos adicionales
Herramientas, plataforma y kernel	9	9
Migrador y paquetes internos	5	5
Plugins, distribución y pruebas	5	5
Infraestructura, docs y generados	12	12
Total	31	31

Siguiente paso técnico

Recrear PostgreSQL con el volumen persistente y probar health, smoke HTTP y persistencia de app. Después podrá comenzar J11-S1-04 con contratos de plugins y validaciones, usando los ejemplos planificados como guía y convirtiéndolos en código probado.

Idea central

migrator.jar protege la transición entre el estado de la base y la versión del ERP. La estructura del repositorio protege los límites entre kernel, plugins, infraestructura y composición. Los ejemplos muestran cómo evolucionar sin volver a crear el acoplamiento del sistema legado.